

Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration and Style-Conditioned Face-to-Sketch Synthesis

Zaid Fareed

Roll No. 23I-0829, Section C — Generative AI, Assignment 1

Source code: https://github.com/Zaid-fareed/23i-0829_C_GenAi_A1

Demonstration video: <https://youtu.be/xUows4eME8M>

Abstract—We build and deploy four generative models. Three restore Oxford-IIIT Pet images corrupted by salt-and-pepper noise, Gaussian blur or occlusion: (1) a universal bottleneck autoencoder, (2) a classifier that routes each image to one of three specialist autoencoders, and (3) a jointly trained soft mixture of experts. The fourth is a style-conditioned cGAN that turns FS2K photographs into sketches in three styles. All hyper-parameters are tuned with Optuna, runs are tracked with MLflow, and the models are exported to ONNX and served by a FastAPI + React application in Docker. On 36,690 test inputs the universal model raises the SSIM of corrupted images from 0.636 to 0.767 but reconstructs clean images at only 0.814. The classifier is 0.9941 accurate; hard routing reaches an overall SSIM of 0.779 and the soft mixture 0.783 (universal: 0.772), the gain coming from clean inputs. The cGAN gives recognisable but soft sketches (SSIM 0.486). We report failure cases and two corrections forced by inspecting outputs: an under-fitted first autoencoder and specialists that lost image colour.

Index Terms—denoising autoencoder, mixture of experts, conditional GAN, Optuna, ONNX, image restoration, face sketch synthesis

I. Introduction

Image restoration recovers a clean image x from a degraded observation $\tilde{x}$. Salt-and-pepper noise destroys isolated pixels, Gaussian blur removes high frequencies everywhere and occlusion removes whole regions, so one network faces very different problems. We study three architectures on the Oxford-IIIT Pet dataset: Task 1, a universal denoising autoencoder with a genuine bottleneck that is never told the corruption; Task 2, a four-class corruption classifier that routes each image to one of three specialist autoencoders (identity bypass for clean inputs); and Task 3, a soft mixture of experts whose gate is initialised from Task 2 and trained jointly. Task 4 is a style-conditioned face-to-sketch cGAN on FS2K. Every task has an Optuna study and MLflow tracking, the models are exported to ONNX and served by a FastAPI + React/Tailwind application in Docker whose interface was designed in Google Stitch. Code, Optuna studies and instructions are at https://github.com/Zaid-fareed/23i-0829_C_GenAi_A1.

II. Related Work and Design Decisions

Autoencoders and losses. Denoising autoencoders learn robust representations by reconstructing a clean input

from a corrupted one [1]. The assignment forbids unrestricted skip connections, so Task 1 keeps the classical bottleneck; the price, measured in Section IV, is that even clean images are reconstructed imperfectly. L1 gives sharper results than L2 but ignores structure, whereas SSIM [2] compares local luminance, contrast and structure; their combination trains restoration networks better than either alone [3], so we use $\alpha\mathcal{L}_1 + (1 - \alpha)(1 - \text{SSIM})$ and let Optuna choose α . SSIM reacts weakly to global colour shifts, which matters for Section V.

Mixture of experts. Gated combinations of specialised networks trained jointly were introduced in [4]; sparsely-gated layers scaled the idea [5]. Task 3 uses a dense softmax gate with a temperature and the squared-deviation balance loss given in the assignment, and starts from trained components instead of random ones.

Conditional image translation. Conditional GANs [6] were applied to paired translation by pix2pix [7], which combines a U-Net generator [8], a PatchGAN discriminator and an L1 term with weight 100. FS2K [9] provides 2,104 photo-sketch pairs in three styles. Pixel losses favour smooth outputs because perception and distortion trade off [10], which explains the softness of our sketches. Tuning uses Optuna [11] (TPE sampler, median pruning), tracking uses MLflow [12].

Main decisions. (i) No skip connections in Tasks 1–3, as required; identity bypasses in Tasks 2 and 3 remove the resulting cost on clean images. (ii) The L1/SSIM weight is searched by Optuna; when the first search chose an SSIM-heavy value that discarded colour, the range was restricted and the models retrained (Section V). (iii) Hard routing and soft mixing are compared on identical inputs. (iv) Non-square photographs for sketches are cropped by default, because stretching distorted faces in a manual test.

III. Data, Corruption Protocol and Metrics

Oxford-IIIT Pet (Tasks 1–3). The official train/validation collection (3,680 images [13]) was split 80/20 with seed 42 into 2,944 training and 736 validation images; the 3,669 official test images were never used for training or tuning. Labels are ignored,

images are converted to RGB and resized (bicubic) to 128×128 ; one split file serves all three tasks.

Runtime corruption. Nothing corrupted is stored. Each time a training image is loaded, one of four conditions (clean, salt-and-pepper, blur, occlusion) is drawn with equal probability and a fresh specification is sampled (Table I). The same function applies it in training, validation and testing; the classifier uses exactly balanced batches ($B/4$ per class).

Manifests. Validation (736 entries, 184 per class) and test manifests are generated once and store the corruption type, severity, parameters, mask coordinates and seed, with a SHA-256 digest. Every test image appears under all ten conditions, giving 36,690 test inputs (3,669 images $\times$ 10); all test tables use this full manifest.

FS2K (Task 4). The official training/test definitions are used. 15% of the training portion is held out as validation, stratified by style with seed 42, leaving 899 training and 159 validation pairs; the 1,046 official test pairs (619, 381 and 46 in styles 1, 2, 3) are used only for the final evaluation. Photographs are 128×128 RGB, sketches 128×128 grayscale; augmentation (flip, random crop-and-resize) is applied identically to both members of a pair.

Metrics. L1 (images in $[0, 1]$), SSIM (11-pixel Gaussian window) and PSNR in dB, clipped at 60 dB because identical images give infinite PSNR. The score of the untouched corrupted input (“no restoration”) is always shown, since a restoration model is useful only where it beats it. Classification uses accuracy, macro precision/recall/F1, per-class values and a row-normalised confusion matrix.

IV. Task 1: Universal Denoising Autoencoder

Architecture. The encoder halves the spatial size four times ($128 \rightarrow 8$) with widths 48, 96, 192, 384 (stride-2 and 3×3 convolutions, batch norm, ReLU); a 1×1 convolution gives a linear latent of $32 \times 8 \times 8 = 2,048$ values (4.2% of the 49,152 pixel values) followed by Dropout2d. The decoder mirrors the encoder with $2 \times$ nearest-neighbour upsampling and ends with a sigmoid. There are no skip connections and no corruption label is given; the model has 4.01 M parameters (Fig. 2).

Loss and training. $\mathcal{L}_{DAE} = \alpha \mathcal{L}_1(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$. Optuna selected $\alpha = 0.58$, learning rate 4.6×10^{-4} and batch 16 (Section VIII). Training uses AdamW, cosine decay, gradient clipping, mixed precision on a T4 and 100 epochs over all 2,944 images with fresh random corruptions; the checkpoint with the lowest validation objective $0.5\mathcal{L}_1 + 0.5(1 - \text{SSIM})$ (epoch 93) is kept.

A first attempt that under-fitted. Our first run (three-epoch trials, 30 epochs on 736 images) was noticeably fast and weak: SSIM 0.545, PSNR 19.55 dB. The train-validation gap was only 0.008, so the model was under-fitting, not memorising, and clean images scored no better than corrupted ones. Optuna had picked dropout 0.29,

which deletes features an autoencoder needs, because very short trials reward fast early learning. We capped dropout at 0.15, lengthened the trials and trained 100 epochs on all data; SSIM on the same manifest rose from 0.545 to 0.779.

Results. Table II separates the results by corruption and severity next to the untouched input. Salt-and-pepper noise is restored well: SSIM $0.382 \rightarrow 0.805$, PSNR $16.4 \rightarrow 25.8$ dB, almost independent of severity. Blur is not helped and clean images are damaged: blur output scores 0.790 SSIM against 0.814 for the input (they tie at $k = 5$), and clean images fall from 1.000 to 0.814, because the bottleneck cannot reproduce fine detail; this motivates Tasks 2 and 3. Occlusion: SSIM and PSNR disagree. SSIM is 0.707 against 0.713 for the masked input, yet PSNR rises from 13.6 to 21.5 dB. SSIM is windowed, so the large unmasked part keeps the input’s score high (doing nothing wins in SSIM on 87% of the 10% masks), while PSNR and Fig. 4 show that the holes are filled with plausible content; only at 35% coverage does restoration also win in SSIM. Generalisation: SSIM on 736 training inputs is 0.797 against 0.783 on unseen validation images (gap 0.014); the curves (Fig. 3) move together.

Failure cases. Figure 5 shows the four worst corrupted inputs, all 35% occlusions. A cat on a black background is split by a mask that merges with the background (SSIM 0.27); three rectangles hide the head and back of a dog on grass (0.31); an L-shaped mask removes a basset hound’s head and the network paints wall and grass instead (0.34); two masks cover a brown dog’s body (0.35). The error sits inside the masks and along the outline: the information is absent, so the best the model can give is a smooth guess that matches the surroundings.

V. Task 2: Classification and Hard-Routed Specialists

Classifier. Four stride-2 stages of two 3×3 convolutions (16–128 channels, batch norm, ReLU, Dropout2d), global average pooling, dropout and a linear layer give $p = [p_{\text{clean}}, p_{\text{salt}}, p_{\text{blur}}, p_{\text{occ}}]$ with 0.295 M parameters and $r = \arg \max_k p_k$. It is trained with multiclass cross-entropy on exactly balanced batches; Optuna tuned learning rate, batch size, channel configuration, dropout and weight decay (objective: validation macro-F1), selecting learning rate 1.5×10^{-3} , batch 32, dropout 0.15, weight decay 1.3×10^{-4} , 30 epochs. On the test protocol it reaches accuracy 0.9941 and macro-F1 0.9902 (Table III, Fig. 7). Salt-and-pepper is recognised perfectly; the 218 errors (0.59%) are clean images taken for blurred or occluded ones, and low-severity occlusions taken for clean images.

Specialists. Three autoencoders A_{salt} , A_{blur} , A_{occ} share the Task 1 architecture but have independent parameters, each trained only on its own corruption with the same clean target. As the assignment allows, one shared Optuna search (each trial trains all three for six epochs on 1,000 images) selects a common configuration (learning rate

TABLE I

Complete corruption configuration. Training values are re-sampled at every load; validation uses the random ranges once with fixed seeds; testing uses three fixed severities.

Condition	Training / validation (random)	Fixed test severities (low, medium, high)
Clean	original image	original image
Salt-and-pepper	$p \sim U(0.02, 0.15)$; exactly $\text{round}(pHW)$ pixels (all channels) set to black or white with equal probability	$p = 0.03, 0.08, 0.15$
Gaussian blur	kernel $\{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$, separable Gaussian, reflect padding	$(k, \sigma) = (3, 0.7), (5, 1.5), (7, 2.5)$
Rectangular occlusion	1–3 black rectangles, union area 10–35% of the image, random positions and aspect ratios	1, 2, 3 rectangles covering 10%, 20%, 35% ($\pm 1\%$)



Fig. 1. The ten deterministic test conditions for one test image (clean and three severities of each corruption).

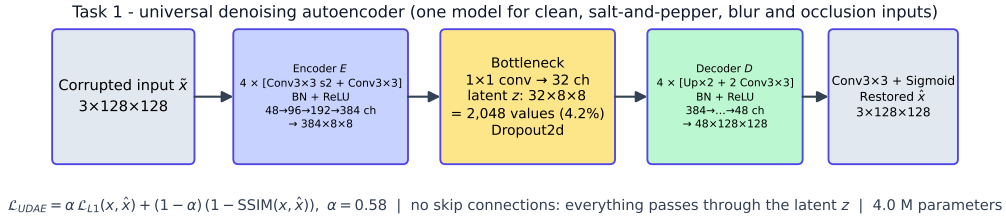


Fig. 2. Universal denoising autoencoder: the only path from input to output is the 2,048-value latent.

TABLE II
Task 1 on the 36,690 test inputs. PSNR is clipped at 60 dB.

Corruption	Setting	Level	Input (no restoration)		Universal autoencoder		
			SSIM	PSNR	L1	SSIM	PSNR
clean	—	—	1.000	≥ 60	0.036	0.814 $\pm$ 0.080	25.97
salt-and-pepper	$p=0.03$	low	0.602	20.13	0.036	0.813 $\pm$ 0.080	25.95
salt-and-pepper	$p=0.08$	medium	0.339	15.87	0.036	0.808 $\pm$ 0.081	25.89
salt-and-pepper	$p=0.15$	high	0.204	13.14	0.037	0.796 $\pm$ 0.082	25.68
Gaussian blur	$k=3, \sigma=0.7$	low	0.944	32.34	0.035	0.814 $\pm$ 0.081	26.17
Gaussian blur	$k=5, \sigma=1.5$	medium	0.806	26.77	0.036	0.805 $\pm$ 0.083	26.04
Gaussian blur	$k=7, \sigma=2.5$	high	0.692	24.35	0.039	0.751 $\pm$ 0.097	25.13
occlusion	1 rect, 10%	low	0.861	16.67	0.043	0.768 $\pm$ 0.083	23.56
occlusion	2 rects, 20%	medium	0.730	13.34	0.051	0.718 $\pm$ 0.086	21.66
occlusion	3 rects, 35%	high	0.547	10.77	0.067	0.634 $\pm$ 0.091	19.21
all corrupted (33,021)			0.636	19.27	0.042	0.767	24.36
all conditions (36,690)			0.672	23.34	0.042	0.772	24.52

6.8×10^{-4} , batch 32, dropout 0.07, $\alpha = 0.60$); the specialists are then trained for 100 epochs. Inference is

$$\hat{x} = \begin{cases} \tilde{x}, & r = \text{clean}, \\ A_{\text{salt}}(\tilde{x}), & r = \text{salt}, \\ A_{\text{blur}}(\tilde{x}), & r = \text{blur}, \\ A_{\text{occ}}(\tilde{x}), & r = \text{occlusion}, \end{cases} \quad (1)$$

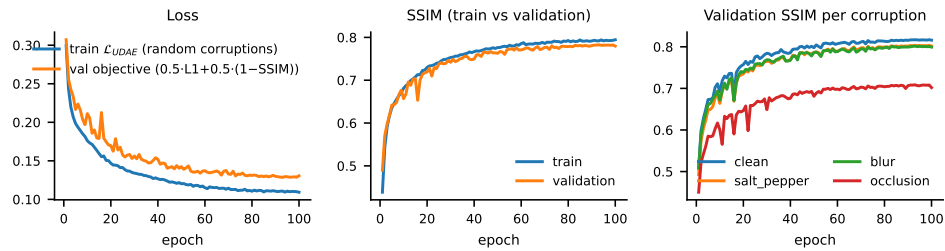


Fig. 3. Task 1 training and validation curves (100 epochs); validation SSIM per corruption.

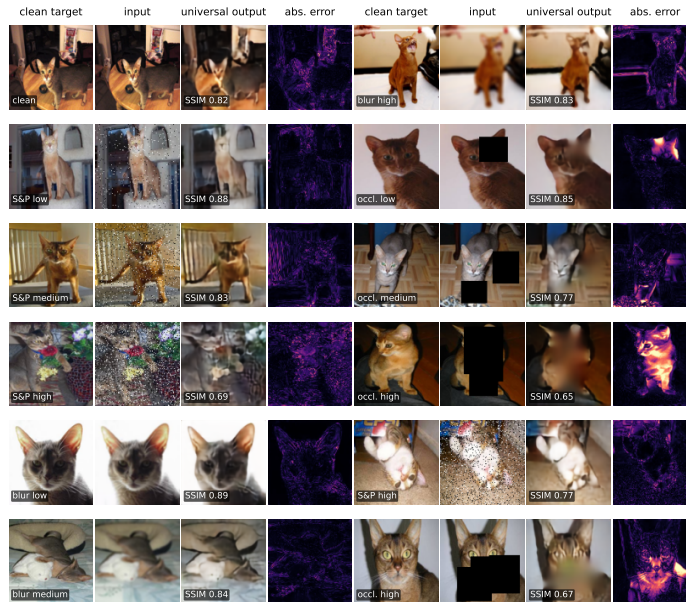


Fig. 4. Twelve test inputs for the universal autoencoder: clean target, input, output with SSIM, and absolute error map.

TABLE III
Classifier metrics on the test protocol.

Class	Support	Precision	Recall	F1
clean	3,669	0.965	0.977	0.971
salt-and-pepper	11,007	1.000	1.000	1.000
Gaussian blur	11,007	0.994	1.000	0.997
occlusion	11,007	0.998	0.988	0.993
macro average	36,690	0.989	0.991	0.990
overall accuracy			0.9941	

TABLE IV
Colour preserved (colourfulness of the output relative to the clean image) on 200 salt-and-pepper and occlusion test inputs.

Model output	salt-and-pepper inputs	occlusion inputs
clean image (reference)	100%	100%
corrupted input	91%	77%
universal AE (Task 1)	97%	96%
specialist, first run ($\alpha \approx 0.3$)	43%	79%
specialist, final ($\alpha \approx 0.6$)	89%	88%
soft MoE, first run ($\alpha \approx 0.3$)	46%	94%
soft MoE, final ($\alpha \approx 0.6$)	97%	94%

so clean inputs take an identity bypass. Oracle routing (true label) measures the specialists; predicted routing measures the deployable system.

Colour loss and its correction. The first specialists (Optuna chose $\alpha = 0.29$) produced washed-out, sepia-toned images, which we noticed when inspecting the outputs. Colourfulness measured on 200 salt-and-pepper test inputs confirmed it (Table IV): the first specialist kept 43% of the colour and the first mixture 46%, the universal model 97%. SSIM is weakly sensitive to a global chroma shift,

so neither the loss nor the Optuna objective penalised it. We restricted α to $[0.55, 0.95]$ and retrained the specialists and the mixture: the final specialist keeps 89% and the mixture 97%; the L1 error of the routed system fell by 18% (0.0456 to 0.0373) with SSIM essentially unchanged. The occlusion specialist still keeps slightly less colour than the universal model (88% against 96%).

Routing results. Predicted routing (SSIM 0.779) is indistinguishable from oracle routing (0.779) because the classifier is accurate (Table V). Its advantage over the

TABLE V

Hard routing on the test protocol: SSIM and L1 of oracle versus predicted routing, and the share of mis-routed inputs.

Input type	SSIM				L1		misroute %
	no restoration	universal	oracle	predicted	oracle	predicted	
clean	1.000	0.814	1.000	0.996	0.0000	0.0010	2.34
salt-and-pepper	0.382	0.805	0.793	0.793	0.0444	0.0444	0.00
Gaussian blur	0.814	0.790	0.780	0.780	0.0449	0.0449	0.01
occlusion	0.713	0.707	0.689	0.690	0.0597	0.0594	1.19
all (36,690)	0.672	0.772	0.779	0.779	0.0447	0.0447	0.59

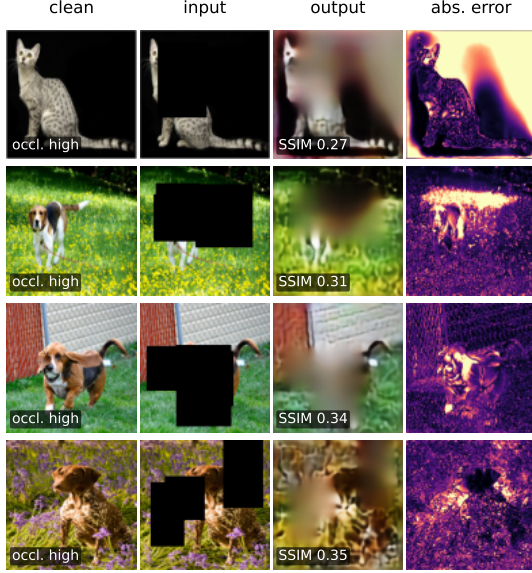


Fig. 5. The four lowest-SSIM corrupted inputs (Task 1).

universal model (0.772) comes entirely from clean images (0.996 against 0.814). On corrupted inputs the specialists are slightly worse than the universal model: 0.793 against 0.805 for noise, 0.780 against 0.790 for blur and 0.690 against 0.707 for occlusion. A specialist has the same capacity and bottleneck as the universal model, so specialisation adds no reconstruction ability, and one shared search for three models and a batch of 32 (half the updates per epoch) may explain the small gap. The measured benefit of routing is the identity bypass.

Classifier-caused failures. The 86 clean images sent to a blur or occlusion expert are real failures (SSIM -0.16 on average relative to the oracle route). In Fig. 9 a clean dog on grass is taken for blurred and “restored” into a smeared brown scene (0.51 instead of 1.00), and an image with black letterbox bars and a cat on a black background are taken for occluded, so the occlusion expert invents content for the dark regions (0.54, 0.57). Conversely, 128 occlusions labelled “clean” (126 of them 10% masks) are left untouched; this scores about +0.12 SSIM higher, but only because SSIM ignores the black rectangle, which

stays visible. Overall, mis-routing changes mean SSIM by only +0.0001 because the two groups cancel. The four worst outputs of the system are, as for Task 1, 35% occlusions that hide the animal (SSIM 0.28–0.33); they are not routing errors, and the occlusion specialist fills the hole with a brownish patch even where the surroundings are green.

VI. Task 3: Jointly Trained Soft Mixture of Experts

Model and loss. A gate G with the classifier’s architecture gives $w = \text{softmax}(G(\tilde{x})/\tau)$ and

$$\hat{x} = w_0 \tilde{x} + w_1 A_{\text{salt}}(\tilde{x}) + w_2 A_{\text{blur}}(\tilde{x}) + w_3 A_{\text{occ}}(\tilde{x}), \quad (2)$$

where branch 0 is the identity (Fig. 6, bottom). Gate and experts are trained jointly through the reconstruction error, starting from the trained Task 2 classifier and the final specialists. A warm-up of 2 epochs trains only the gate with the experts frozen; 30 epochs of joint fine-tuning with a much smaller learning rate follow. The loss is

$$\mathcal{L}_{MoE} = \lambda_1 \mathcal{L}_1 + \lambda_s (1 - \text{SSIM}) + \lambda_c \mathcal{L}_{CE} + \lambda_b \sum_{k=0}^3 (\bar{w}_k - \frac{1}{4})^2, \quad (3)$$

with $\bar{w}_k$ the mean weight of branch k in a batch and the cross-entropy applied to the raw gate logits. We tie $\lambda_1 = \alpha$, $\lambda_s = 1 - \alpha$, so the five tuned quantities (joint learning rate, temperature, λ_c , λ_b , α) match the assignment’s list. Optuna selected $\tau = 2.68$, $\lambda_c = 0.48$, $\lambda_b = 1.20$, $\alpha = 0.61$ and learning rate 4.3×10^{-5} ; median pruning removed half of the trials. The first mixture inherited the colour loss of its experts and was retrained from the corrected specialists (Section V).

Reconstruction. The mixture reaches an overall SSIM of 0.783, against 0.779 for hard routing and 0.772 for the universal model (Table VI, Fig. 12). Clean images stay almost untouched (0.991) because the gate puts most of its weight on the identity branch. On corrupted types it matches the other systems (noise 0.793, blur 0.790, occlusion 0.697); it is slightly better than hard routing at low severity (blur $k = 3$: 0.820 against 0.792) and slightly worse at high severity (noise $p = 0.15$: 0.778 against 0.790), as a less committed decision would be. Blending does not remove the limit on heavy occlusion.

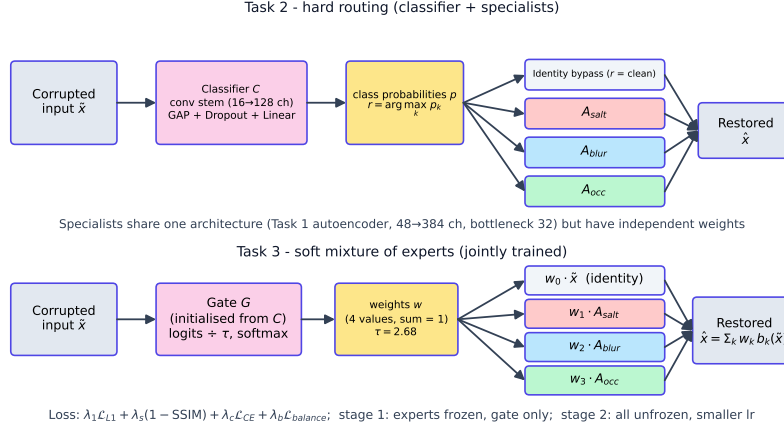


Fig. 6. Top: hard routing (Task 2); the classifier output r selects a specialist or the identity bypass. Bottom: the soft mixture of experts (Task 3) built from the same components.

TABLE VI
Soft mixture (Task 3) against the universal model and predicted hard routing on the test protocol.

Corruption	Level	SSIM			soft MoE	
		universal	hard (pred.)	soft MoE	L1	PSNR (dB)
clean	—	0.814	0.996	0.991 ± 0.017	0.0081	40.19
salt-and-pepper	low	0.813	0.796	0.807 ± 0.070	0.0427	24.98
salt-and-pepper	medium	0.808	0.794	0.795 ± 0.075	0.0436	24.83
salt-and-pepper	high	0.796	0.790	0.778 ± 0.075	0.0441	24.71
Gaussian blur	low	0.814	0.792	0.820 ± 0.074	0.0400	25.55
Gaussian blur	medium	0.805	0.790	0.795 ± 0.086	0.0424	25.04
Gaussian blur	high	0.751	0.757	0.754 ± 0.096	0.0448	24.49
occlusion	low	0.768	0.747	0.771 ± 0.083	0.0484	22.78
occlusion	medium	0.718	0.699	0.699 ± 0.093	0.0575	21.28
occlusion	high	0.634	0.625	0.621 ± 0.098	0.0718	19.16
all (36,690)		0.772	0.779	0.783 ± 0.120	0.0443	25.30

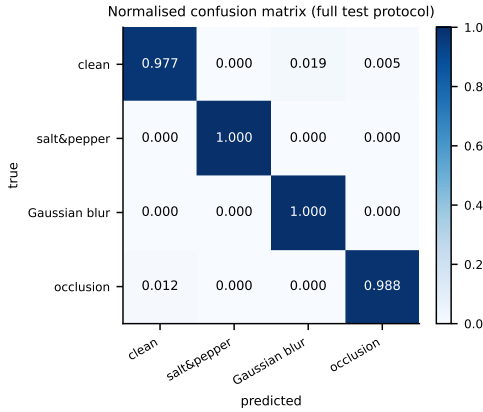


Fig. 7. Row-normalised confusion matrix of the classifier.

Gate behaviour. Figure 13 gives the mean weight of each branch for every true corruption and severity. The matching expert receives on average 0.86 of the weight

for salt-and-pepper, 0.85 for blur and 0.92 for occlusion, and the identity branch 0.81 for clean images; the weight grows with severity (occlusion: 0.80 at 10% to 0.99 at 35% coverage). Weights are spread rather than one-hot because of the temperature (mean largest weight 0.868). Figure 14 contrasts inputs on which one expert dominates with inputs on which the weights are distributed; the latter are low-severity corruptions that resemble clean images (mean weight entropy 0.58 nats for 10% occlusions and 0.64 for clean images, against 0.06 for 35% occlusions). No expert is inactive and none dominates unrelated inputs: each branch is the top branch for a substantial share of inputs (identity 10.5%, salt-and-pepper 30.0%, blur 30.1%, occlusion 29.4%).

Failure cases. The four worst outputs (SSIM 0.24–0.31, Fig. 16) are again 35% occlusions. In the first, a black mask merges with the cat’s black background and the mixture produces a grey-brown cloud; in the other three (beagle, basset hound, dark terrier on grass) the gate correctly gives the occlusion expert nearly all the weight,

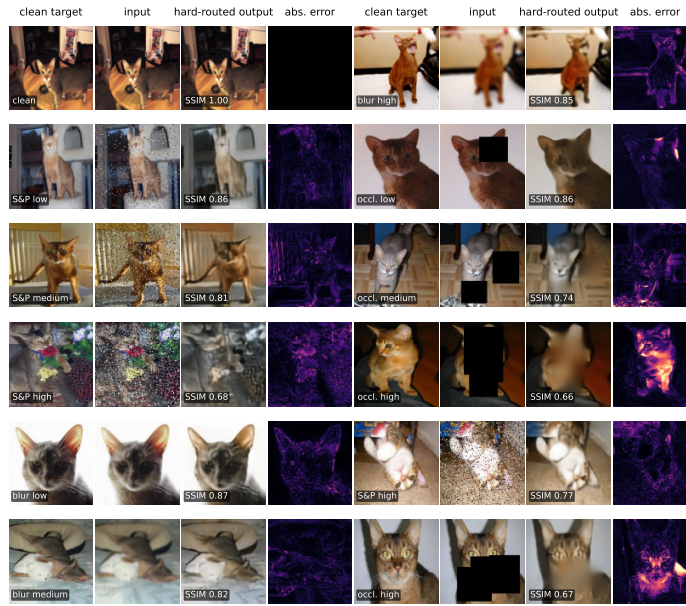


Fig. 8. Twelve test inputs for hard-routed restoration (predicted routing).

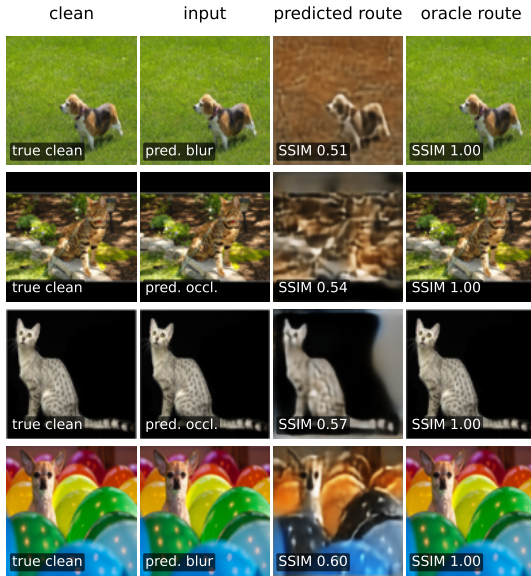


Fig. 9. Classifier-induced failures: the four mis-routed clean images with the largest SSIM loss; predicted route next to oracle route.

but the expert cannot recreate the animal. They are 0.03–0.04 SSIM below the worst universal cases; because the information is absent, no routing or blending strategy recovers it.

VII. Task 4: Style-Conditioned Face-to-Sketch cGAN

Model. The generator $\hat{y} = G(x, s)$ is a U-Net [8]: seven stride-2 encoder blocks (64–512 channels) reduce the 128×128 photograph to a 1×1 code, six transposed-convolution blocks with skip connections restore the res-

olution, and a final tanh layer gives a one-channel sketch (dropout 0.30 in the first three decoder blocks, 42.1M parameters, Fig. 17). The style $s \in \{1, 2, 3\}$ is a learned categorical embedding (three vectors of 32 values) that enters the generator twice, as extra input channels and concatenated to the bottleneck, and the discriminator as extra input channels, so it is part of the model and not an interface label. The PatchGAN discriminator [7] receives photograph, style map and a real or generated sketch and outputs a 14×14 map of patch logits (2.8M parameters).

Losses and training. The discriminator uses binary cross-entropy on logits (real pairs one, generated pairs zero; both losses are recorded), and the generator minimises $\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_{L1}(y, G(x, s))$; both use Adam ($\beta_1 = 0.5$). Optuna tuned both learning rates, batch size, base channels, dropout, embedding size and λ_{L1} with 15-epoch trials (three completed, five pruned). The selected configuration (generator learning rate 1.5×10^{-4} , discriminator 8.6×10^{-4} , batch 8, 64 base channels, dropout 0.30, embedding 32, $\lambda_{L1} = 121$) was trained for 100 epochs, logging discriminator real/fake loss, generator adversarial and L1 loss, and validation L1/SSIM/PSNR per epoch (Fig. 18). Every ten epochs the same eight validation photographs are rendered in all three styles and logged to MLflow (Fig. 19).

Results. On the 1,046 official test pairs the model reaches L1 0.1051, SSIM 0.486 and PSNR 15.76 dB (Table VII, Fig. 20). Style 2 has the largest error (L1 0.152 against 0.079 for style 1 and 0.066 for style 3) because its reference sketches use heavy dark strokes that a soft generator cannot match pixel by pixel; style-3 figures rest on only 46 pairs. The same photograph is rendered visibly differently in each style (Fig. 21): style 2 darkest, style 1

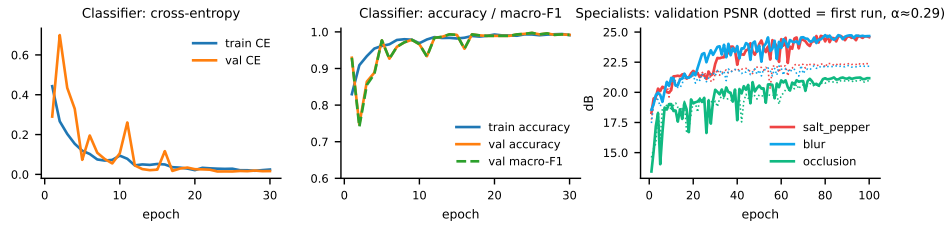


Fig. 10. Task 2 training curves: classifier loss and accuracy, and validation PSNR of the three specialists in the first (dotted) and final (solid) run.

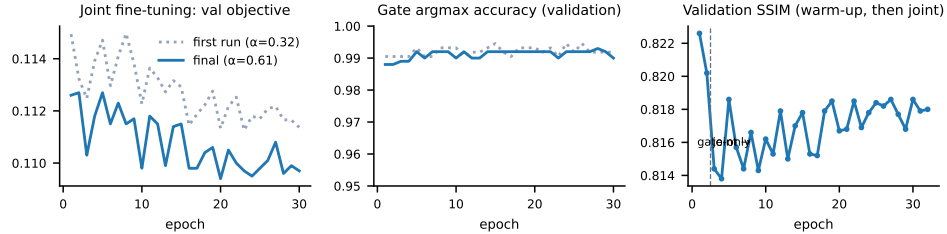


Fig. 11. Task 3 training: validation objective and gate accuracy for the first (dotted) and final run, and validation SSIM over the warm-up and joint stages.

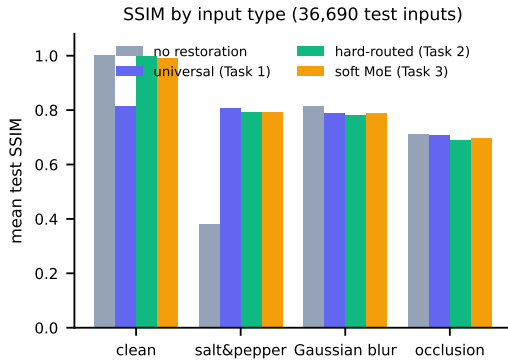


Fig. 12. Mean test SSIM per input type: corrupted input, universal, hard-routed and soft-mixture models.

TABLE VII

Task 4 on the FS2K test set. The last row is a second recipe (photo-only jitter, learning-rate decay) that was tried and rejected.

Test subset	Pairs	L1	SSIM	PSNR (dB)
Style 1	619	0.0791	0.534 $\pm$ 0.107	17.55
Style 2	381	0.1522	0.395 $\pm$ 0.082	12.47
Style 3	46	0.0656	0.600 $\pm$ 0.074	18.90
all (selected model)	1046	0.1051	0.486	15.76
all (rejected variant)	1046	0.1112	0.451	15.23

lightest.

Why the sketches are soft. The average gradient magnitude of the generated sketches is only 37% of that of the real ones (measured on the test set). A sharper input helps a little and a blurred one hurts a little, but the effect is small next to this gap, so the limit lies in the

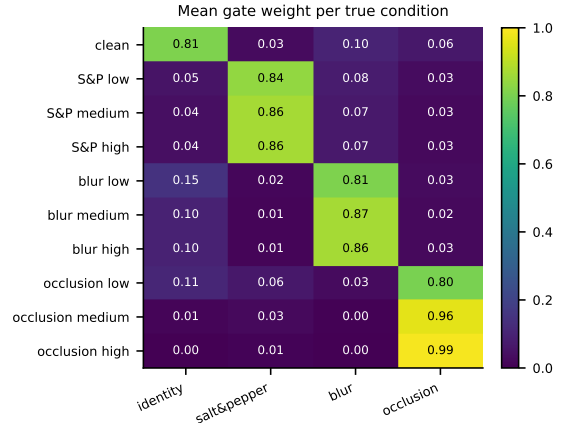


Fig. 13. Routing heat-map: mean weight of each branch by true corruption and severity.

training and not in the 128×128 input. Two causes fit the evidence. With 899 training pairs and $\lambda_{L1} = 121$ the L1 term dominates and regresses to an average stroke; and in the selected Optuna trial the discriminator stayed near chance (loss near $\ln 2$), while in the 100-epoch run it did learn to separate real from fake (losses about 0.4, generator adversarial loss about 2) but the L1 term still dominates. This is the perception-distortion trade-off [10]. A second recipe did not help (SSIM 0.451, L1 0.1112) and was discarded; stronger adversarial or perceptual losses and more data were not explored.

Failure cases. The four lowest-SSIM test pairs (Fig. 21) are portraits with long hair: three in style 2, where the reference draws hair with dense dark strokes and the

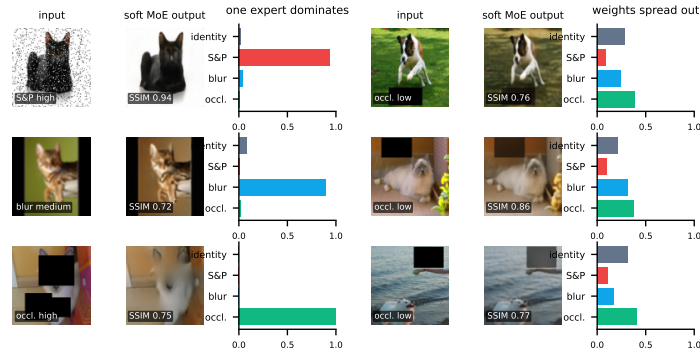


Fig. 14. Left: inputs on which a single expert dominates. Right: inputs with weights spread over several branches.

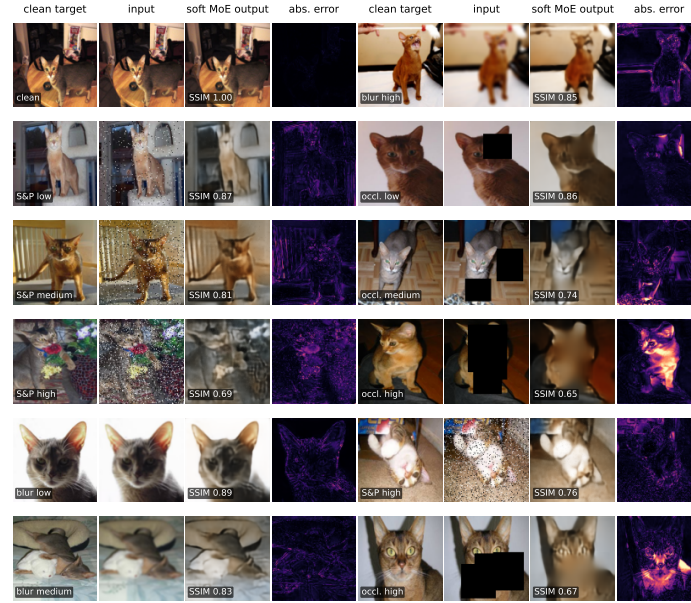


Fig. 15. Twelve test inputs for the soft mixture of experts.

generator a smooth grey mass (SSIM 0.18–0.20), and one in style 1 with thin crisp reference lines and a pale, blurred output (0.21). The faces remain recognisable, so these are failures of stroke detail, not identity. A thin line missing or shifted by a few pixels costs a large pixel error, which pixel metrics punish heavily on hair-rich style 2 subjects.

VIII. Hyper-parameter Optimisation with Optuna

Each task has its own Optuna study (TPE sampler, seed 42, SQLite files in `optuna_studies/`, every trial also logged to MLflow). Trials are short and use sub-samples of the training data; the selected configuration is retrained on all data. Table VIII lists the search spaces, Table IX the trial counts and best trials, Table X the final configurations and Fig. 22 the study histories. Median pruning stopped weak trials in Tasks 1, 2 (classifier), 3 and 4; the shared specialist search is not pruned because each trial trains three models.

Three lessons stand out. Short trials favour configurations that learn fast early, which is how the first Task 1 search picked a high dropout that later under-fitted. An L1+SSIM objective cannot see colour loss, so the first specialist search picked an SSIM-heavy α ; narrowing a range after inspecting outputs is acceptable only when reported, as here. With 8–12 trials per study the searches find good, not optimal, settings.

IX. Application, Deployment and Tracking

Interface design. The interface was designed first in Google Stitch [14]. A first prompt described four workspaces with a sidebar, input and corruption cards, result panels and a system-information area; Stitch returned four screens and a design system (indigo, Inter, white rounded cards). Using them, we found that the controls and Run button sat below the fold, so a run needed scrolling down to choose options and back up to see the result. A second prompt asked for a layout where

TABLE VIII

Optuna search spaces (log = log-uniform, U = uniform, $\{\cdot\}$ = categorical). Channel options c16, c32, c48 are the widths (16, ..., 128), (32, ..., 256), (48, ..., 384).

Study	Searched parameters	Trials and objective
Task 1 (final)	lr $\log[3e-4, 3e-3]$; batch $\{16, 32, 64\}$; latent channels $\{32, 64, 128, 256\}$; channels $\{c16, c32, c48\}$; dropout $U[0, 0.15]$; α $U[0.2, 0.95]$	12×6 epochs, 1,000 images; $0.5\mathcal{L}_1 + 0.5(1 - \text{SSIM})$
Task 1 (first)	as above, dropout $U[0, 0.3]$, latent $\{16, 32, 64, 128\}$	8×3 epochs, 600 images
Task 2 classifier	lr $\log[3e-4, 3e-3]$; batch $\{16, 32, 64\}$; channels $\{c16, c32, c48\}$; dropout $U[0, 0.5]$; weight decay $\log[10^{-6}, 10^{-2}]$	8×4 epochs, 1,000 images; macro-F1
Task 2 specialists	as Task 1; α $U[0.2, 0.95]$ (first) and $U[0.55, 0.95]$ (final)	8 trials, each training the three specialists 6 epochs; mean objective
Task 3	joint lr $\log[10^{-5}, 5e-4]$; τ $\log[0.3, 3]$; λ_c $\log[0.01, 2]$; λ_b $U[0, 2]$; α as Task 2	$8 \times (1 \text{ warm-up} + 3 \text{ joint epochs})$; objective
Task 4	lr_G, lr_D $\log[5e-5, 10^{-3}]$; batch $\{8, 16, 32\}$; base channels $\{32, 48, 64\}$; dropout $U[0, 0.5]$; embedding $\{8, 16, 32\}$; λ_{L1} $\log[10, 200]$	8×15 epochs, 899 pairs; $0.5\mathcal{L}_1 + 0.5(1 - \text{SSIM})$



Fig. 16. The four lowest-SSIM corrupted inputs (soft mixture).

TABLE IX

Optuna studies: trials, completed and pruned counts and best trial (“first” and “rejected” studies are kept as evidence).

Study	Trials	Completed	Pruned	Best trial	Best value
Task 1 universal AE	12	6	6	#9	0.2622
Task 2 classifier	8	4	4	#1	0.9243
Task 2 specialists (first run)	8	8	0	#6	0.2826
Task 2 specialists (final, $\alpha \geq 0.55$)	8	8	0	#6	0.2816
Task 3 soft MoE (first run)	8	4	4	#0	0.1135
Task 3 soft MoE (final, $\alpha \geq 0.55$)	8	4	4	#0	0.1108
Task 4 cGAN (selected)	8	3	5	#0	0.3139
Task 4 cGAN (rejected variant)	12	5	7	#0	0.3077

everything for one run fits on one laptop screen (narrow control panel with a pinned Run button, large results on the right). Both rounds and the project page are shown in Fig. 23; the prompts are in `stitch_design/PROMPTS.md`.

TABLE X
Selected configuration of each model.

Model	Selected configuration
Task 1	lr=4.61e-04, batch_size=16, bottleneck_dim=32, channels=c48, dropout=1.04e-03, alpha=0.583
Classifier	lr=1.53e-03, batch_size=32, channels=c16, dropout=0.152, weight_decay=1.26e-04
Specialists	lr=6.85e-04, batch_size=32, bottleneck_dim=32, channels=c48, dropout=0.071, alpha=0.598
Soft MoE	lr=4.33e-05, temperature=2.678, w_cls=0.483, w_bal=1.197, alpha=0.612
cGAN	lr_g=1.54e-04, lr_d=8.63e-04, batch_size=8, base=64, dropout=0.301, emb_dim=32, lambda_l1=121.069

The exported HTML holds invented data (GPU model, LPIPS), so only its structure, colours and typography were re-implemented as React components that show real data.

Front end and back end. The React 18 / Vite / Tailwind front end has four workspaces (Fig. 24): the user uploads an image, picks a built-in test sample or (for sketches) uses the webcam, chooses a corruption and severity (or custom sliders and a seed) or none, and presses Run. Results show the model input, output, absolute error map, inference time and corruption settings; hard routing adds the four classifier probabilities, the predicted corruption, the selected expert and oracle/predicted mode; the mixture adds the four weights, a stacked contribution bar and the dominant branch; sketches show photograph and sketch side by side with a download button. A browser measurement showed that at 1366×768 and 1920×1080 the Run button is always visible and the page does not scroll. The FastAPI service loads the seven ONNX models once and offers `/api/health`, `/api/restore/universal`, `/hard`, `/soft` and `/api/sketch`. It validates uploads (type, 10 MB

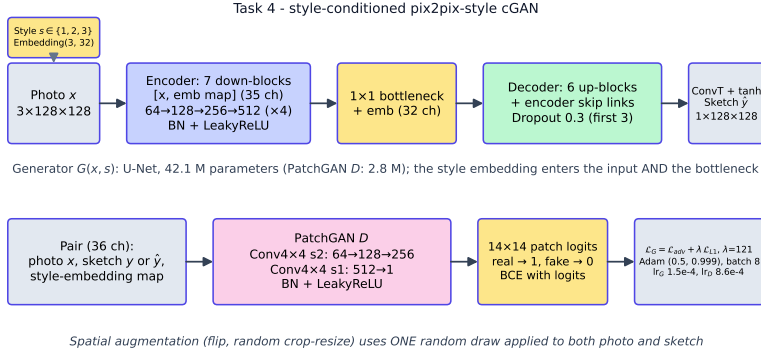


Fig. 17. Style-conditioned cGAN: U-Net generator with a style embedding in the input and bottleneck, and PatchGAN discriminator.

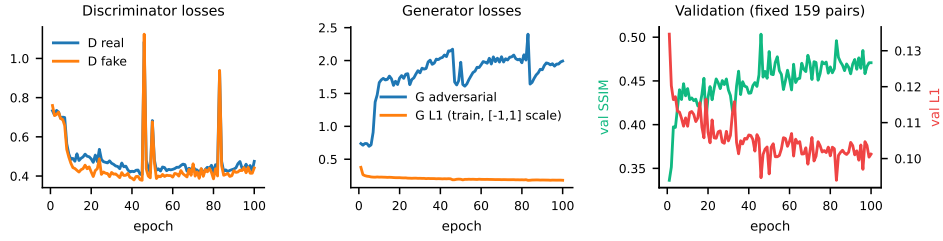


Fig. 18. Task 4 training: discriminator and generator losses, validation SSIM and L1.

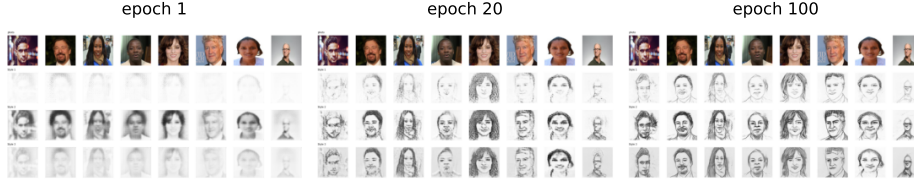


Fig. 19. The same eight validation photographs (top row of each panel) rendered in styles 1, 2, 3 after epochs 1, 20 and 100.

and 40 megapixel limits, decodability, style and slider ranges), preprocesses exactly as in training, applies corruptions with a NumPy port of the training code, runs ONNX Runtime on the CPU and returns images, routing information and timings; a missing model gives a clear 503 message instead of a crash.

ONNX. All inference models were exported (opset 17) and compared with PyTorch on a random batch; the maximum absolute differences are all below 2.1×10^{-6} (Table XI). The soft mixture (gate, experts, identity and mixing) is one ONNX graph returning the image and the four weights; only the cGAN generator is exported. The reported test results were also re-measured using only the deployed ONNX files, the back end’s corruptions and a different SSIM implementation; SSIM agreed to four decimals on the first 300 test entries.

Deployment. Two containers are built from the repository, the back end (Python, FastAPI, ONNX Runtime) and the front end (nginx serving the build and

TABLE XI
ONNX files: size, maximum absolute difference to PyTorch, CPU latency per image.

Model	Size (MB)	Max PyTorch-ONNX	Latency (ms)
Task 1 universal autoencoder	16.1	5.4e-7	12.4
Task 2 classifier	1.2	2.0e-6	0.3
Task 2 specialist (salt-and-pepper)	16.1	5.7e-7	15.3
Task 2 specialist (blur)	16.1	4.5e-7	12.4
Task 2 specialist (occlusion)	16.1	5.4e-7	15.3
Task 3 soft mixture (whole pipeline)	49.4	2.4e-7	39.8
Task 4 generator	168.5	1.4e-6	21.1

proxying /api). The model files are mounted from ./onnx_models, are not stored in Git and are fetched by scripts/download_models.py from the models-v1 release of the repository. After cloning, docker compose up – build starts the application at http://localhost:3000; an optional profile starts MLflow on port 5000. A smoke test exercises every endpoint, option and error path through the nginx address (54 checks), and the back end has 38 unit tests. README.md and KAGGLE_GUIDE.md give

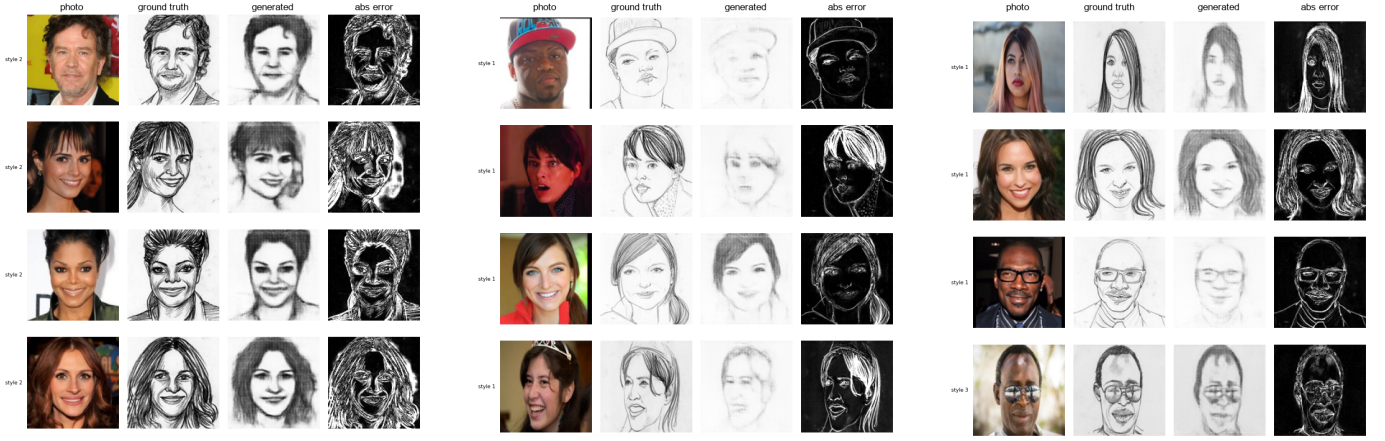


Fig. 20. Twelve test pairs spread over the test set: photograph, ground-truth sketch, generated sketch and absolute error (left to right in each strip).



Fig. 21. Left: one photograph in the three styles. Right: the four test pairs with the lowest SSIM (0.18–0.21).

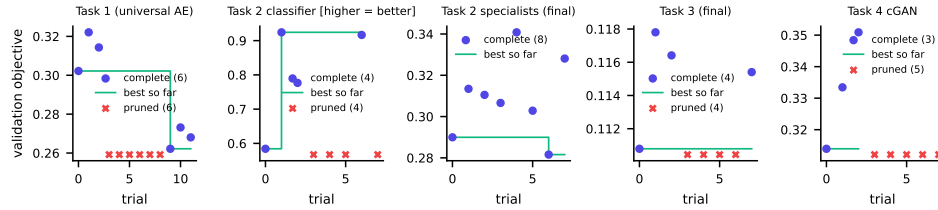


Fig. 22. Optuna studies of the final configurations: completed trials (dots), best value so far (line) and pruned trials (crosses).

the complete steps to run, retrain and evaluate.

Experiment tracking. Runs, Optuna trials, hyper-parameters, per-epoch losses and metrics, checkpoints and visual outputs (Task 4 sample grids every ten epochs) are recorded with MLflow in five experiments; the MLflow database and Optuna studies are in the repository (Fig. 25).

X. Limitations and Conclusion

- The universal model damages inputs that need no restoration (clean SSIM 0.814); routing and mixing exist to avoid this, and specialisation itself did not improve corrupted cases (0.01–0.02 SSIM worse).

- SSIM rewards leaving small occlusions untouched, although PSNR is about 8 dB lower and the hole is visible, so occlusion conclusions rest on PSNR and visual inspection too. Heavy occlusion is irreversible: outputs in masked regions are plausible blur.
- Small searches (8–12 trials, three completed for Task 4) find good, not optimal, settings; alternative GAN objectives were not tried. The α range of the final specialists and mixture was restricted after looking at the outputs.
- Task 3 batches are balanced only in expectation; the classifier used exactly balanced batches.
- Evaluation history. Kaggle scripts used a single-

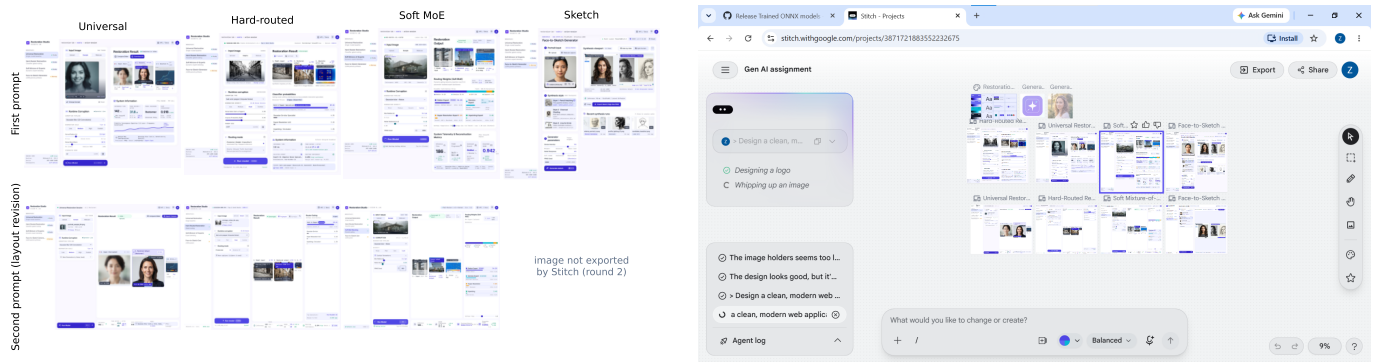


Fig. 23. Left: Stitch designs of the first prompt (top) and the layout revision (bottom). Right: the Stitch project page with both rounds and the prompt list.

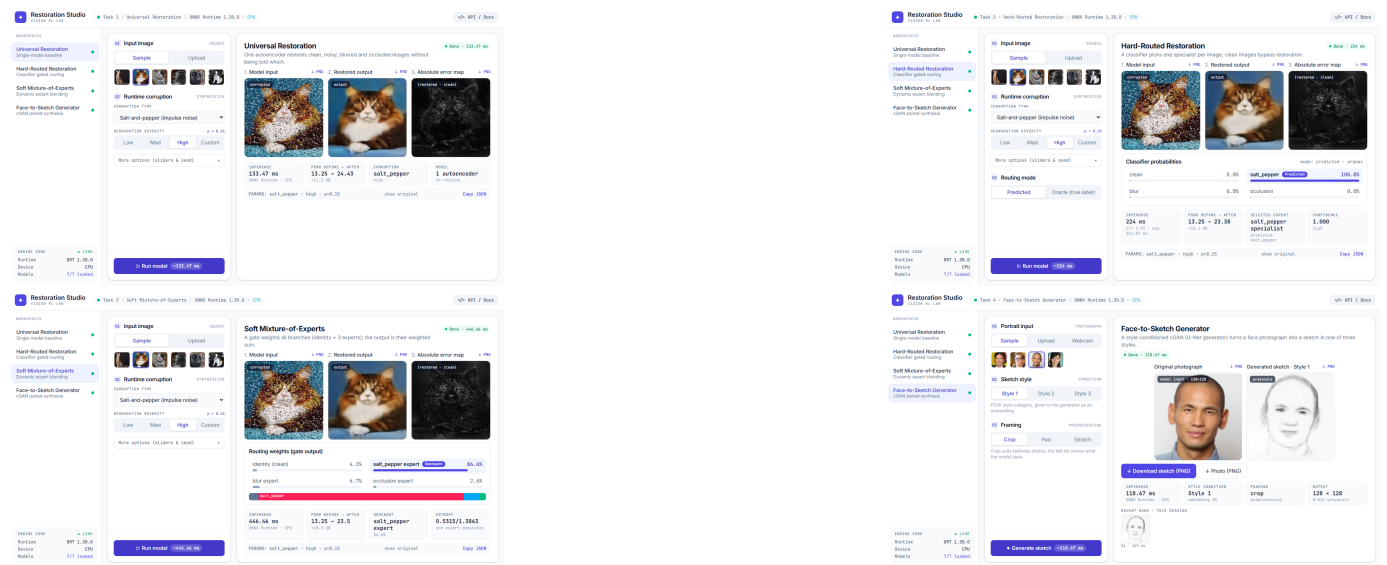


Fig. 24. The deployed application (1366 × 768): universal, hard-routed, soft mixture and face-to-sketch workspaces after a run on a built-in sample.



Fig. 25. MLflow: the experiments with their runs (left) and the metrics of one run (right).

condition test manifest (3,669 inputs); the full protocol (36,690 inputs) was evaluated afterwards on the deployed ONNX models because the final training checkpoints were not archived. Results are from one training run per configuration, without confidence intervals.

- Domain. The restoration models saw only cats and dogs and restore other subjects (such as faces) poorly; the cGAN saw frontal, near-square photographs, so photographs should be cropped.
- Soft sketches: 37% of the real edge strength, and only 46 test pairs in style 3. The CPU inference was not

load-tested.

Conclusion. A single bottleneck autoencoder restores salt-and-pepper noise well (SSIM $0.382 \rightarrow 0.805$) and fills occlusions plausibly (PSNR $13.6 \rightarrow 21.5$ dB) but damages clean and mildly blurred images. A classifier with an identity bypass removes that damage (clean SSIM $0.814 \rightarrow 0.996$); the jointly trained mixture does the same (0.991) without an inactive expert, and overall SSIM moves from 0.772 to 0.779 and 0.783, entirely through clean inputs. Two failures found by inspecting outputs (an under-fitted first model, specialists that lost colour) and the SSIM/PSNR disagreement on occlusion show that aggregate metrics and Optuna objectives are not enough. The cGAN produces recognisable, style-dependent but soft sketches. Everything runs from one docker compose up with verified ONNX models and tracked experiments.

Appendix A AI-Use Statement

AI assistants were used for this assignment and are disclosed here with the tasks they supported and how their output was checked.

A. Tools and tasks

- Claude (web chat, Anthropic): wrote the initial project prompt and a first code base for Tasks 1–3 (data pipeline, corruptions, models, losses, Optuna and training scripts, evaluation and ONNX export scripts).
- Claude Code (Anthropic): integrated and debugged that code, wrote the Task 4 cGAN, the FastAPI back end, the React/Tailwind front end, the Docker set-up, the Kaggle instructions, the evaluation, figure and table scripts, and a draft of this report, which was then revised and shortened at my request.
- Google Stitch: generated the interface design from two prompts (Section IX); its HTML was a structural reference only.
- Kaggle notebooks (T4 GPU) for all training; standard libraries (PyTorch, Optuna, MLflow, ONNX Runtime, FastAPI, React, Tailwind, Docker).

B. Testing and corrections

- My own manual testing drove the main changes. I ran the training on Kaggle and inspected the outputs and the application myself. Noticing that the first training run was suspiciously fast and its output poor led to the diagnosis and retraining of the autoencoder (Section IV); noticing washed-out colours in the specialists' outputs led to the colourfulness measurement, the restricted loss-weight range and the retraining of the specialists and mixture (Section V); my concern about how non-square photographs were framed for sketches led to a test and to a crop option; and the first Stitch interface was hard to use (scrolling

between options and results), which led to the second Stitch prompt and the revised layout.

- Automated checks written with the coding assistant supported this work: tests of corruption determinism and pixel counts, balanced batches, resume-from-checkpoint, the application's endpoints (38 unit tests and a 54-check smoke test), and ONNX-versus-PyTorch comparison of every export (differences $\leq 2.1 \times 10^{-6}$). They supplement my manual testing and do not replace it.
- Numbers and references. Every table value in this report is generated from saved result files by the scripts in report/. The bibliographic details of the references were checked against the publishers' or arXiv records.

References

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in Proc. Int. Conf. Machine Learning (ICML), 2008, pp. 1096–1103.
- [2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.
- [3] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," IEEE Trans. Computational Imaging, vol. 3, no. 1, pp. 47–57, 2017.
- [4] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," Neural Computation, vol. 3, no. 1, pp. 79–87, 1991.
- [5] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Int. Conf. Learning Representations (ICLR), 2017.
- [6] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv preprint arXiv:1411.1784, 2014.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 5967–5976.
- [8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241.
- [9] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. V. Gool, "Facial-sketch synthesis: A new challenge," Machine Intelligence Research, vol. 19, no. 4, pp. 257–287, 2022.
- [10] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 6228–6237.
- [11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2019, pp. 2623–2631.
- [12] MLflow contributors, "MLflow: an open source platform for the machine learning lifecycle," <https://mlflow.org>.
- [13] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3498–3505.
- [14] Google, "Stitch — design user interfaces with AI," <https://stitch.withgoogle.com>.